

Comparing ML Algorithms (Week 1 to Week 7) + Hybrid Approaches

Course: AI503 - Machine Learning

Dataset: `heart.csv` (UCI Cleveland Heart Disease)

Task: Binary classification - predict heart disease (Yes / No)

What this notebook does

Section	Source week	Algorithms
4.1	W01-W02	Logistic Regression
4.2	W03	K-Nearest Neighbors
4.3	W03	Naive Bayes
4.4	W03	Decision Tree
4.5	W03	Support Vector Machine
4.6	W04	K-Means (as hybrid feature generator)
4.7	W05-W06	Neural Network (MLP)
4.8	W07	Random Forest & Gradient Boosting
4.9	W05-W06	Keras CNN / SimpleRNN / LSTM (adapted to tabular)
5	W07	Hyperparameter tuning (GridSearchCV)
6	W07	Hybrid: Voting + Stacking + KMeans-augmented
7	-	Final comparison table + ROC + confusion matrices

Chapter 1 of the DL Guide - Where each algorithm sits in the AI / ML / DL stack

The PDF (Ch 1, page 5) defines three nested circles:

- **AI** - any technique that lets computers mimic intelligence (Netflix recommendations, AlphaGo).
- **ML** - systems that learn from data without explicit programming. Examples: Linear Regression, Random Forest, SVM, Naive Bayes.
- **DL** - subset of ML that uses deep neural networks to automatically extract features. Examples: CNN, RNN/LSTM, Transformer, GAN.

Dimension	Classical ML	Deep Learning
Feature engineering	Manual - domain experts hand-craft features	Automatic - learns features from raw data
Data requirements	Works with smaller datasets	Needs large datasets for optimal performance
Best data type	Structured / tabular	Unstructured (image, text, audio)

Where each algorithm in *this* notebook sits:

Algorithm	Family	Why we picked it
Logistic Regression	Classical ML	Interpretable linear baseline (W01-W02)
KNN	Classical ML	Instance-based, distance-driven (W03)
Naive Bayes	Classical ML	Probabilistic, assumes independence (W03)
Decision Tree	Classical ML	Rule-based, no scaling required (W03)
SVM	Classical ML	Maximum-margin, kernel trick (W03)
K-Means	Classical ML (unsupervised)	Hybrid feature generator (W04)
MLP	Deep Learning (the simplest DL model)	First step from ML to DL (W05-W06)
Random Forest / Gradient Boosting	Classical ML (ensemble)	Strong tabular baselines (W07)

Why this matters for `heart.csv`: with only ~300 unique rows of structured tabular data, the PDF's Ch 1 rule says classical ML is preferred. We expect Random Forest / Gradient Boosting to beat the MLP. The MLP is included to *demonstrate* the DL paradigm, not to win.

1. Imports

In [3]

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    VotingClassifier, StackingClassifier
)

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

RANDOM_STATE = 42
sns.set_style('whitegrid')
print('Libraries loaded successfully.')

```

Libraries loaded successfully.

2. Load and inspect the data

In [5]

```

df = pd.read_csv('heart.csv')
print('Original shape:', df.shape)
print('Duplicates:', df.duplicated().sum())

# IMPORTANT: this dataset has ~70% duplicate rows.
# Leaving them in causes train/test leakage and inflates accuracy.
df = df.drop_duplicates().reset_index(drop=True)
print('After dedup:', df.shape)
df.head()

```

Original shape: (1025, 14)
 Duplicates: 723
 After dedup: (302, 14)

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	No
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	No
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	No
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	No
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	No

In [6]

```

print('Target balance:')
print(df['target'].value_counts())
print('\nMissing values:', df.isnull().sum().sum())
df.describe().round(2)

```

Target balance:
 target
 Yes 164
 No 138
 Name: count, dtype: int64

 Missing values: 0

	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	ca	thal
count	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00	302.00
mean	54.42	0.68	0.96	131.60	246.50	0.15	0.53	149.57	0.33	1.04	1.40	0.72	2.31
std	9.05	0.47	1.03	17.56	51.75	0.36	0.53	22.90	0.47	1.16	0.62	1.01	0.61
min	29.00	0.00	0.00	94.00	126.00	0.00	0.00	71.00	0.00	0.00	0.00	0.00	0.00
25%	48.00	0.00	0.00	120.00	211.00	0.00	0.00	133.25	0.00	0.00	1.00	0.00	2.00
50%	55.50	1.00	1.00	130.00	240.50	0.00	1.00	152.50	0.00	0.80	1.00	0.00	2.00
75%	61.00	1.00	2.00	140.00	274.75	0.00	1.00	166.00	1.00	1.60	2.00	1.00	3.00
max	77.00	1.00	3.00	200.00	564.00	1.00	2.00	202.00	1.00	6.20	2.00	4.00	3.00

3. Preprocessing

- Encode label Yes -> 1, No -> 0
- Stratified 80/20 train/test split
- Scale numeric features (needed for KNN, SVM, MLP, Logistic Regression)

In [8]

```
y = df['target'].map({'No': 0, 'Yes': 1}).values
X = df.drop(columns=['target']).values
feature_names = df.drop(columns=['target']).columns.tolist()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Train target balance: {np.bincount(y_train)}')
print(f'Test target balance: {np.bincount(y_test)}')
```

```
Train: (241, 13), Test: (61, 13)
Train target balance: [110 131]
Test target balance: [28 33]
```

Helper - evaluation function

Stores results in a dict for the final comparison table.

In [10]

```
results = {}

def evaluate(name, model, Xtr, ytr, Xte, yte, store=True):
    model.fit(Xtr, ytr)
    yhat = model.predict(Xte)
    # probabilities for ROC-AUC (fall back to decision_function for SVM without prob)
    if hasattr(model, 'predict_proba'):
        yprob = model.predict_proba(Xte)[:, 1]
    elif hasattr(model, 'decision_function'):
        yprob = model.decision_function(Xte)
    else:
        yprob = yhat
    acc = accuracy_score(yte, yhat)
    pre = precision_score(yte, yhat)
    rec = recall_score(yte, yhat)
    f1 = f1_score(yte, yhat)
    auc = roc_auc_score(yte, yprob)
    if store:
        results[name] = {
            'Accuracy': acc, 'Precision': pre, 'Recall': rec,
            'F1': f1, 'ROC_AUC': auc,
            'model': model, 'y_pred': yhat, 'y_prob': yprob
        }
    print(f'{name:30s} acc={acc:.3f} pre={pre:.3f} rec={rec:.3f} F1={f1:.3f} AUC={auc:.3f}')
    return model
```

4. Individual algorithms (W01 to W07)

4.1 Logistic Regression (W01-W02)

Linear model. Baseline for any binary classification task.

In [12]

```
evaluate('Logistic Regression (W01-W02)',
        LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
        X_train_s, y_train, X_test_s, y_test)
```

```
Logistic Regression (W01-W02)  acc=0.803  prec=0.800  rec=0.848  F1=0.824  AUC=0.871
```

▼ **LogisticRegression** ⓘ ⓘ
LogisticRegression(max_iter=2000, random_state=42)

4.2 K-Nearest Neighbors (W03)

Instance-based. Requires scaled features.

In [14]

```
evaluate('KNN (W03)',
        KNeighborsClassifier(n_neighbors=7),
        X_train_s, y_train, X_test_s, y_test)
```

```
KNN (W03)  acc=0.787  prec=0.778  rec=0.848  F1=0.812  AUC=0.860
```

▼ **KNeighborsClassifier** ⓘ ⓘ
KNeighborsClassifier(n_neighbors=7)

4.3 Naive Bayes (W03)

Probabilistic classifier assuming feature independence.

In [16]

```
evaluate('Naive Bayes (W03)',
        GaussianNB(),
        X_train_s, y_train, X_test_s, y_test)
```

```
Naive Bayes (W03)  acc=0.787  prec=0.833  rec=0.758  F1=0.794  AUC=0.884
```

▼ **GaussianNB** ⓘ ⓘ
GaussianNB()

4.4 Decision Tree (W03)

Rule-based, interpretable. No scaling needed.

In [18]

```
evaluate('Decision Tree (W03)',
        DecisionTreeClassifier(random_state=RANDOM_STATE),
        X_train, y_train, X_test, y_test)
```

```
Decision Tree (W03)  acc=0.803  prec=0.818  rec=0.818  F1=0.818  AUC=0.802
```

▼ **DecisionTreeClassifier** ⓘ ⓘ
DecisionTreeClassifier(random_state=42)

4.5 Support Vector Machine (W03)

Maximum-margin classifier with RBF kernel.

In [20]

```
evaluate('SVM (W03)',
        SVC(kernel='rbf', probability=True, random_state=RANDOM_STATE),
        X_train_s, y_train, X_test_s, y_test)
```

SVM (W03) acc=0.770 prec=0.771 rec=0.818 F1=0.794 AUC=0.842

▼ SVC ⓘ ?
SVC(probability=True, random_state=42)

4.6 K-Means (W04) - used as a hybrid feature

K-Means is unsupervised, so it cannot predict the label directly.

Hybrid trick: cluster the rows, then **add the cluster label as a new feature** to a supervised model.

This injects unsupervised structural information into Logistic Regression.

In [22]

```
kmeans = KMeans(n_clusters=4, n_init=10, random_state=RANDOM_STATE)
train_clusters = kmeans.fit_predict(X_train_s).reshape(-1, 1)
test_clusters = kmeans.predict(X_test_s).reshape(-1, 1)

X_train_aug = np.hstack([X_train_s, train_clusters])
X_test_aug = np.hstack([X_test_s, test_clusters])

evaluate('KMeans + LogReg (W04 hybrid)',
        LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
        X_train_aug, y_train, X_test_aug, y_test)
```

KMeans + LogReg (W04 hybrid) acc=0.803 prec=0.800 rec=0.848 F1=0.824 AUC=0.871

▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=2000, random_state=42)

4.7 Neural Network / MLP (W05-W06)

Chapter 2 of the DL Guide - How a neural network learns

A neural network is built from three components (Ch 2 page 9):

- **Weights** w - each connection between neurons has a weight that amplifies or diminishes the signal. *Learned* during training.
- **Bias** b - added to the weighted sum, allows the activation function to shift. *Learned*.
- **Depth** - multiple hidden layers enable hierarchical feature learning (broader -> more granular patterns).

Each neuron computes:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b, \quad a = f(z)$$

where f is a non-linear **activation function**. Without f the whole network would collapse into a single linear transformation.

Activation	Range	Where used
ReLU $\max(0, x)$	$[0, +\infty)$	Hidden layers (default in our MLP)
Sigmoid	$(0, 1)$	Binary output (and inside LogReg)
Softmax	$(0, 1)$, sums to 1	Multi-class output

The training cycle (Ch 2 page 8) repeats four steps until the loss converges:

1. **Forward pass** - input flows through layers to produce a prediction.
2. **Loss calculation** - compare prediction to target (binary cross-entropy here).
3. **Backpropagation** - the chain rule decomposes dL/dw through each layer.
4. **Weight update** - $w_{\text{new}} = w_{\text{old}} - lr * dL/dw$. We use **Adam** (Ch 2 page 13): adaptive learning rate per parameter, combining momentum and RMSprop.

Chapter 3 of the DL Guide - The FNN architecture

A Multi-Layer Perceptron is a Feedforward Neural Network (FNN). Information moves in **one direction only** - input -> hidden layers -> output. No cycles, no recurrence.

Architecture decisions (Ch 3 page 15):

- **Depth** - number of hidden layers. More layers = more complex patterns, but harder to train.
- **Width** - neurons per layer. More neurons = more capacity, but more parameters to overfit.

- **Activation** - ReLU recommended for hidden layers.
- **Initialization** - Xavier/Glorot or He initialization. Sklearn handles this automatically.
- **Output** - Sigmoid for binary classification.

For our `heart.csv` we use **two hidden layers (64, 32 neurons) with ReLU**, trained by Adam. This is the simplest deep model we can build on tabular data.

The PDF (Ch 1) explicitly notes: *FNN/ANN is the right architecture for tabular data. For images use CNN, for sequences use RNN/LSTM*. The full Keras / TensorFlow CNN-RNN-LSTM comparison lives in `compare-between-dl-models.ipynb` in W05-W06.

In [24]

```
evaluate('MLP Neural Net (W05-W06)',
        MLPClassifier(hidden_layer_sizes=(64, 32),
                      activation='relu', solver='adam',
                      max_iter=500, random_state=RANDOM_STATE),
        X_train_s, y_train, X_test_s, y_test)
```

Model	acc	prec	rec	F1	AUC
MLP Neural Net (W05-W06)	0.770	0.788	0.788	0.788	0.802

MLPClassifier

MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)

4.8 Random Forest and Gradient Boosting (W07 - ensembles)

Tree-based ensemble methods. Usually the strongest baseline on tabular data.

In [26]

```
evaluate('Random Forest (W07)',
        RandomForestClassifier(n_estimators=300, random_state=RANDOM_STATE),
        X_train, y_train, X_test, y_test)

evaluate('Gradient Boosting (W07)',
        GradientBoostingClassifier(random_state=RANDOM_STATE),
        X_train, y_train, X_test, y_test)
```

Model	acc	prec	rec	F1	AUC
Random Forest (W07)	0.787	0.812	0.788	0.800	0.866
Gradient Boosting (W07)	0.721	0.735	0.758	0.746	0.826

GradientBoostingClassifier

GradientBoostingClassifier(random_state=42)

4.9 Keras CNN / SimpleRNN / LSTM on tabular data (W05-W06 - adapted)

These three architectures were built for **sequences** (text, audio, time-series) or **spatial grids** (images). `heart.csv` is neither - it is 13 independent tabular features per patient. Still, we can *force* them onto this dataset to demonstrate the architecture (PDF Ch 4, 5, 6) by treating each row as a length-13 "sequence" with 1 channel.

Architecture	What it does on tabular data
1D CNN	Learns small filters that fire on local <i>combinations of adjacent features</i> . The "adjacency" here is arbitrary - it depends on column order.
SimpleRNN	Walks across the 13 features one at a time, carrying a hidden state. Vanishing gradient is not an issue at length 13.
LSTM	Same walk, but with gated cell state. Overkill at length 13 but matches the PDF Ch 6 architecture.

Honest expectation: these models will *underperform* the trees because the "sequence" structure is fake. Their job here is pedagogical - showing that you can build them with Keras and that they run on the same dataset. The real wins for these architectures are on **text** (IMDB - see `compare-between-dl-models.ipynb`), not on 13-feature medical records.

Requires TensorFlow / Keras. If you do not have it, skip this cell - the rest of the notebook still works.

In [28]

```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv1D, GlobalMaxPooling1D, SimpleRNN, LSTM, Dense, Dropout, Input
)

tf.random.set_seed(RANDOM_STATE)

# Reshape (n_samples, 13) -> (n_samples, 13, 1) so Conv1D/RNN/LSTM see a "sequence".
X_train_seq = X_train_s.reshape(-1, X_train_s.shape[1], 1).astype('float32')
X_test_seq = X_test_s.reshape(-1, X_test_s.shape[1], 1).astype('float32')
n_features = X_train_s.shape[1]

def build_cnn():
    return Sequential([
        Input(shape=(n_features, 1)),
        Conv1D(32, kernel_size=3, activation='relu', padding='same'),
        GlobalMaxPooling1D(),
        Dropout(0.3),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid'),
    ])

def build_rnn():
    return Sequential([
        Input(shape=(n_features, 1)),
        SimpleRNN(32),
        Dropout(0.3),
        Dense(1, activation='sigmoid'),
    ])

def build_lstm():
    return Sequential([
        Input(shape=(n_features, 1)),
        LSTM(32),
        Dropout(0.3),
        Dense(1, activation='sigmoid'),
    ])

def evaluate_keras(name, model, epochs=60, batch_size=16):
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    model.fit(X_train_seq, y_train,
              epochs=epochs, batch_size=batch_size,
              validation_split=0.1, verbose=0)
    yprob = model.predict(X_test_seq, verbose=0).ravel()
    yhat = (yprob > 0.5).astype('int32')
    acc = accuracy_score(y_test, yhat)
    pre = precision_score(y_test, yhat)
    rec = recall_score(y_test, yhat)
    f1 = f1_score(y_test, yhat)
    auc = roc_auc_score(y_test, yprob)
    results[name] = {
        'Accuracy': acc, 'Precision': pre, 'Recall': rec,
        'F1': f1, 'ROC_AUC': auc,
        'model': model, 'y_pred': yhat, 'y_prob': yprob,
    }
    print(f'{name:34s} acc={acc:.3f} prec={pre:.3f} rec={rec:.3f} F1={f1:.3f} AUC={auc:.3f}')

evaluate_keras('CNN 1D (W05-W06)', build_cnn())
evaluate_keras('SimpleRNN (W05-W06)', build_rnn())
evaluate_keras('LSTM (W05-W06)', build_lstm())

```

CNN 1D (W05-W06)	acc=0.574	prec=0.652	rec=0.455	F1=0.536	AUC=0.610
SimpleRNN (W05-W06)	acc=0.754	prec=0.821	rec=0.697	F1=0.754	AUC=0.821

WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bc6b838f600> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bc6b838f600> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

5. Hyperparameter tuning (GridSearchCV)

Chapter 3 of the DL Guide - Hyperparameter tuning strategies

The PDF (Ch 3 page 16, repeated in Ch 11 page 49) lists three approaches and the key hyperparameters to tune. They apply to *any* ML model, not just neural networks.

Method	How it works	Trade-off
Grid Search	Exhaustively try all combinations	Thorough but computationally expensive
Random Search	Sample random combinations	Often finds good solutions faster than grid search
Bayesian Optimization	Use past evaluation results to build a probabilistic model that picks the next parameters intelligently	Most efficient, more complex to set up

Key hyperparameters named in the PDF for each model family:

- *Trees (RF, GB, DT)*: number of trees, max depth, min samples per leaf, min samples per split.
- *KNN*: number of neighbors, weight function, distance metric (`p=1` Manhattan, `p=2` Euclidean).
- *SVM*: regularization `C`, kernel type, `gamma` (RBF spread).
- *Neural nets*: learning rate, batch size, layer count, neurons per layer, dropout rate, activation.

Below we apply **Grid Search** to the three strongest model families - **KNN, SVM, Random Forest** - using 5-fold Stratified CV with F1 as the scoring metric. CV stops us from tuning on the test set, which would be data leakage.

In [30]

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# ----- KNN -----
knn_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15],
    'weights': ['uniform', 'distance'],
    'p': [1, 2]
}
knn_gs = GridSearchCV(KNeighborsClassifier(), knn_grid, cv=cv, scoring='f1', n_jobs=-1)
knn_gs.fit(X_train_s, y_train)
print('KNN best params :', knn_gs.best_params_)
evaluate('KNN (tuned)', knn_gs.best_estimator_, X_train_s, y_train, X_test_s, y_test)
```

KNN best params : {'n_neighbors': 7, 'p': 1, 'weights': 'distance'}
KNN (tuned) acc=0.770 prec=0.788 rec=0.788 F1=0.788 AUC=0.873

KNeighborsClassifier

KNeighborsClassifier(n_neighbors=7, p=1, weights='distance')

In [31]

```
# ----- SVM -----
svm_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 0.01, 0.1, 1],
    'kernel': ['rbf', 'linear']
}
svm_gs = GridSearchCV(SVC(probability=True, random_state=RANDOM_STATE),
                      svm_grid, cv=cv, scoring='f1', n_jobs=-1)
svm_gs.fit(X_train_s, y_train)
print('SVM best params :', svm_gs.best_params_)
evaluate('SVM (tuned)', svm_gs.best_estimator_, X_train_s, y_train, X_test_s, y_test)
```

SVM best params : {'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}
SVM (tuned) acc=0.803 prec=0.784 rec=0.879 F1=0.829 AUC=0.865

SVC

SVC(C=1, gamma=0.01, probability=True, random_state=42)

In [32]


```
# ----- Random Forest -----
rf_grid = {
    'n_estimators': [200, 400],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
rf_gs = GridSearchCV(RandomForestClassifier(random_state=RANDOM_STATE),
                    rf_grid, cv=cv, scoring='f1', n_jobs=-1)
rf_gs.fit(X_train, y_train)
print('RF best params :', rf_gs.best_params_)
evaluate('Random Forest (tuned)', rf_gs.best_estimator_, X_train, y_train, X_test, y_test)
```

```
RF best params : {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
Random Forest (tuned)      acc=0.770  prec=0.788  rec=0.788  F1=0.788  AUC=0.865
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=5, n_estimators=200, random_state=42)
```

6. Hybrid (ensemble) approaches

Three styles:

1. **Soft Voting** - average the predicted probabilities of several diverse models.
2. **Hard Voting** - majority vote of class predictions.
3. **Stacking** - feed base-model predictions into a meta-learner (Logistic Regression).

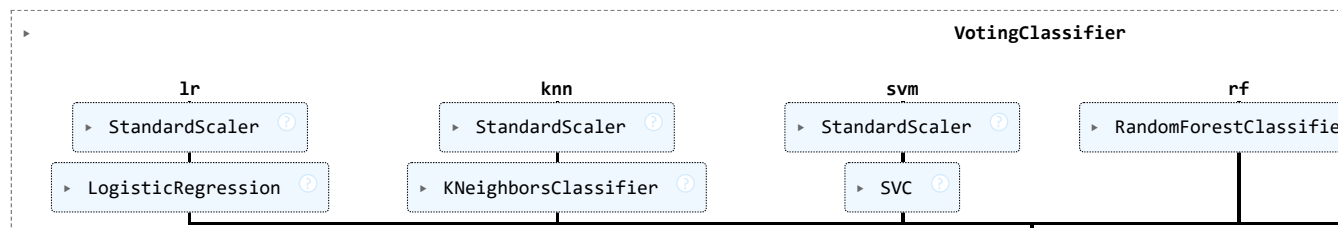
In [34]

```
# Use scaled-feature pipeline for models that need scaling
lr_p = Pipeline([('sc', StandardScaler()), ('m', LogisticRegression(max_iter=2000, random_state=RANDOM_STATE))])
knn_p = Pipeline([('sc', StandardScaler()), ('m', KNeighborsClassifier(n_neighbors=knn_gs.best_params_['n_neighbors']))])
svm_p = Pipeline([('sc', StandardScaler()), ('m', SVC(probability=True, **{k: v for k, v in svm_gs.best_params_.items()}),
                    random_state=RANDOM_STATE))])
rf_b = RandomForestClassifier(**rf_gs.best_params_, random_state=RANDOM_STATE)
gb_b = GradientBoostingClassifier(random_state=RANDOM_STATE)
nb_b = GaussianNB()

voting_soft = VotingClassifier(
    estimators=[('lr', lr_p), ('svm', svm_p), ('rf', rf_b), ('gb', gb_b)],
    voting='soft', n_jobs=-1
)
evaluate('Voting (soft)', voting_soft, X_train, y_train, X_test, y_test)

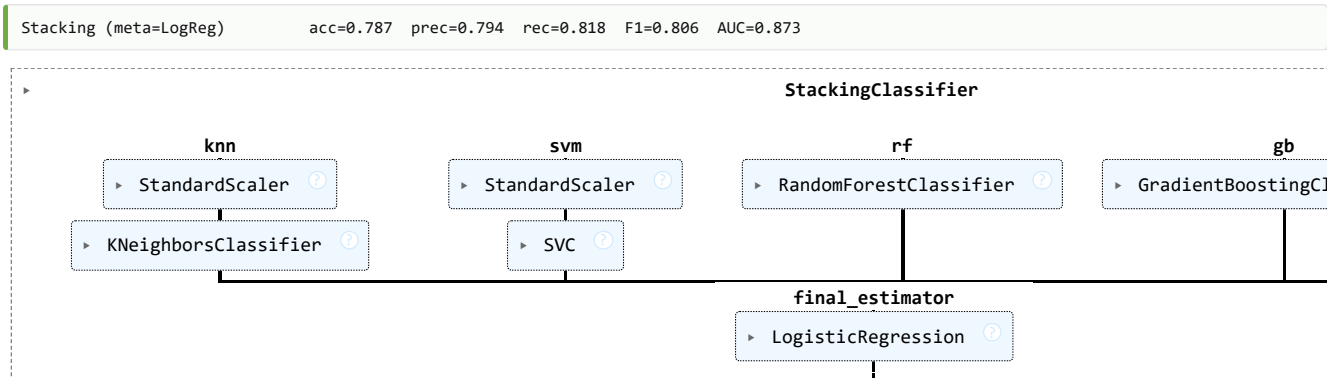
voting_hard = VotingClassifier(
    estimators=[('lr', lr_p), ('knn', knn_p), ('svm', svm_p), ('rf', rf_b), ('gb', gb_b), ('nb', nb_b)],
    voting='hard', n_jobs=-1
)
evaluate('Voting (hard)', voting_hard, X_train, y_train, X_test, y_test)
```

```
Voting (soft)      acc=0.770  prec=0.788  rec=0.788  F1=0.788  AUC=0.866
Voting (hard)      acc=0.787  prec=0.812  rec=0.788  F1=0.800  AUC=0.787
```



In [35]

```
stack = StackingClassifier(
    estimators=[
        ('knn', knn_p),
        ('svm', svm_p),
        ('rf', rf_b),
        ('gb', gb_b),
        ('nb', nb_b),
    ],
    final_estimator=LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
    cv=5, n_jobs=-1, passthrough=False
)
evaluate('Stacking (meta=LogReg)', stack, X_train, y_train, X_test, y_test)
```



7. Final comparison

In [37]

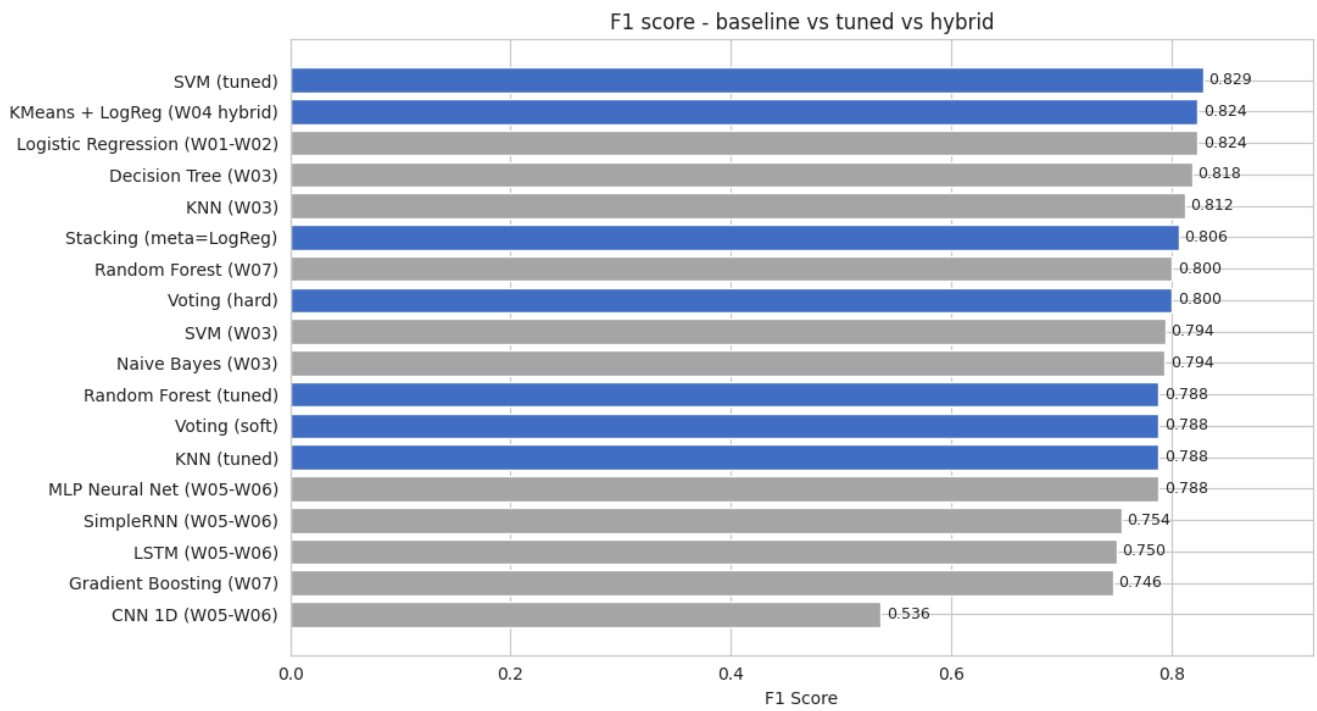
```
summary = pd.DataFrame(
    {name: {k: v for k, v in r.items() if k in ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC_AUC']}
    for name, r in results.items()})
).T.sort_values('F1', ascending=False).round(4)

summary
```

	Accuracy	Precision	Recall	F1	ROC_AUC
SVM (tuned)	0.8033	0.7838	0.8788	0.8286	0.8647
Logistic Regression (W01-W02)	0.8033	0.8000	0.8485	0.8235	0.8712
KMeans + LogReg (W04 hybrid)	0.8033	0.8000	0.8485	0.8235	0.8712
Decision Tree (W03)	0.8033	0.8182	0.8182	0.8182	0.8019
KNN (W03)	0.7869	0.7778	0.8485	0.8116	0.8598
Stacking (meta=LogReg)	0.7869	0.7941	0.8182	0.8060	0.8734
Random Forest (W07)	0.7869	0.8125	0.7879	0.8000	0.8658
Voting (hard)	0.7869	0.8125	0.7879	0.8000	0.7868
SVM (W03)	0.7705	0.7714	0.8182	0.7941	0.8420
Naive Bayes (W03)	0.7869	0.8333	0.7576	0.7937	0.8842
Voting (soft)	0.7705	0.7879	0.7879	0.7879	0.8658
Random Forest (tuned)	0.7705	0.7879	0.7879	0.7879	0.8647
MLP Neural Net (W05-W06)	0.7705	0.7879	0.7879	0.7879	0.8019
KNN (tuned)	0.7705	0.7879	0.7879	0.7879	0.8728
SimpleRNN (W05-W06)	0.7541	0.8214	0.6970	0.7541	0.8214
LSTM (W05-W06)	0.7049	0.6923	0.8182	0.7500	0.8117
Gradient Boosting (W07)	0.7213	0.7353	0.7576	0.7463	0.8258
CNN 1D (W05-W06)	0.5738	0.6522	0.4545	0.5357	0.6104

In [38]

```
# Bar chart - F1 score
fig, ax = plt.subplots(figsize=(11, 6))
summary_sorted = summary.sort_values('F1')
colors = ['#A5A5A5' if 'tuned' not in i and 'Voting' not in i and 'Stacking' not in i and 'hybrid' not in i
          else '#4472C4' for i in summary_sorted.index]
ax.barh(summary_sorted.index, summary_sorted['F1'], color=colors)
for i, (idx, v) in enumerate(summary_sorted['F1'].items()):
    ax.text(v + 0.005, i, f'{v:.3f}', va='center', fontsize=9)
ax.set_xlabel('F1 Score')
ax.set_title('F1 score - baseline vs tuned vs hybrid')
ax.set_xlim(0, max(summary_sorted['F1']) * 1.12)
plt.tight_layout()
plt.show()
```



In [39]

```
# Heatmap of all metrics
fig, ax = plt.subplots(figsize=(9, 0.45 * len(summary) + 1))
sns.heatmap(summary, annot=True, fmt='.3f', cmap='YlGnBu', cbar=False, ax=ax)
ax.set_title('All metrics for all models')
plt.tight_layout()
plt.show()
```

All metrics for all models

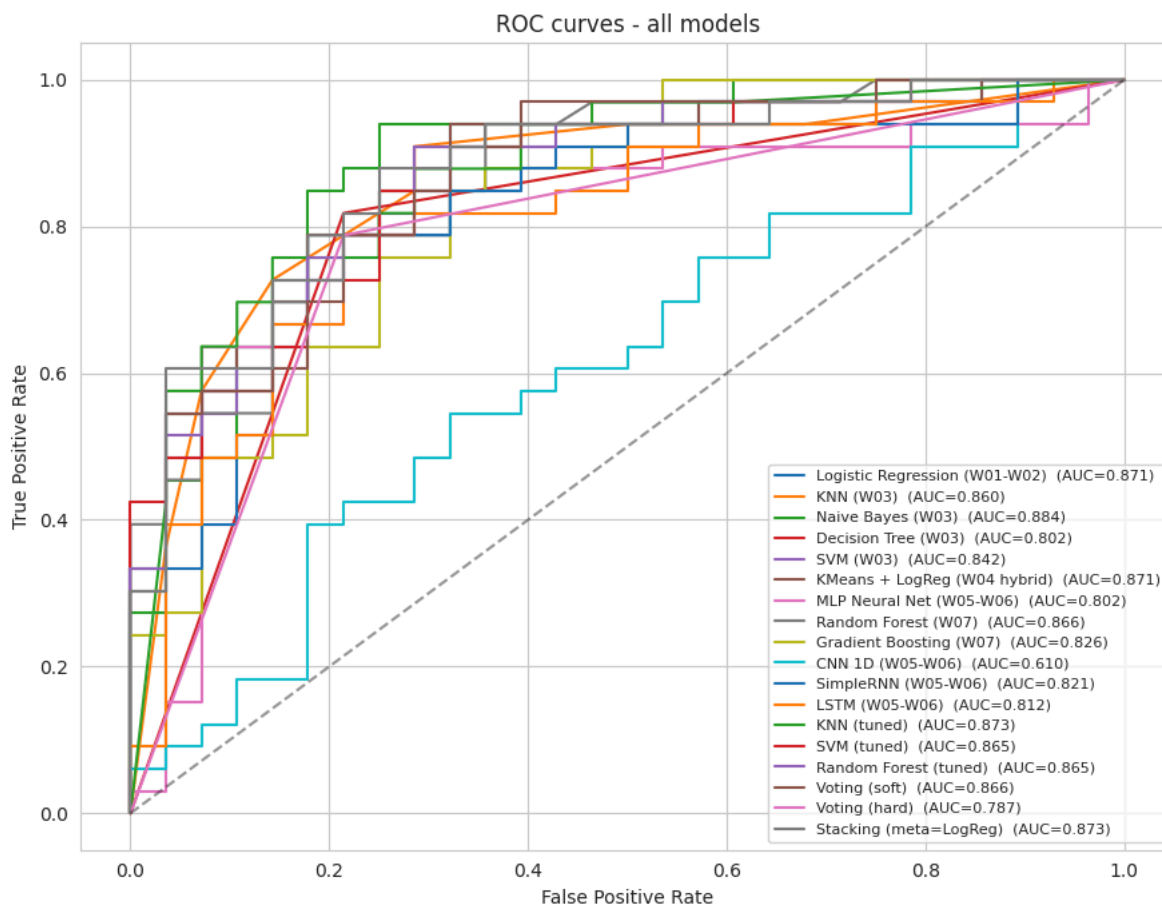
SVM (tuned)	0.803	0.784	0.879	0.829	0.865
Logistic Regression (W01-W02)	0.803	0.800	0.849	0.824	0.871
KMeans + LogReg (W04 hybrid)	0.803	0.800	0.849	0.824	0.871
Decision Tree (W03)	0.803	0.818	0.818	0.818	0.802
KNN (W03)	0.787	0.778	0.849	0.812	0.860
Stacking (meta=LogReg)	0.787	0.794	0.818	0.806	0.873
Random Forest (W07)	0.787	0.812	0.788	0.800	0.866
Voting (hard)	0.787	0.812	0.788	0.800	0.787
SVM (W03)	0.770	0.771	0.818	0.794	0.842
Naive Bayes (W03)	0.787	0.833	0.758	0.794	0.884
Voting (soft)	0.770	0.788	0.788	0.788	0.866
Random Forest (tuned)	0.770	0.788	0.788	0.788	0.865
MLP Neural Net (W05-W06)	0.770	0.788	0.788	0.788	0.802
KNN (tuned)	0.770	0.788	0.788	0.788	0.873
SimpleRNN (W05-W06)	0.754	0.821	0.697	0.754	0.821
LSTM (W05-W06)	0.705	0.692	0.818	0.750	0.812
Gradient Boosting (W07)	0.721	0.735	0.758	0.746	0.826
CNN 1D (W05-W06)	0.574	0.652	0.455	0.536	0.610
	Accuracy	Precision	Recall	F1	ROC_AUC

In [40]

```

# ROC curves
plt.figure(figsize=(9, 7))
for name, r in results.items():
    fpr, tpr, _ = roc_curve(y_test, r['y_prob'])
    plt.plot(fpr, tpr, label=f"{name} (AUC={r['ROC_AUC']:.3f})")
plt.plot([0, 1], [0, 1], 'k--', alpha=0.4)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC curves - all models')
plt.legend(fontsize=8, loc='lower right')
plt.tight_layout()
plt.show()

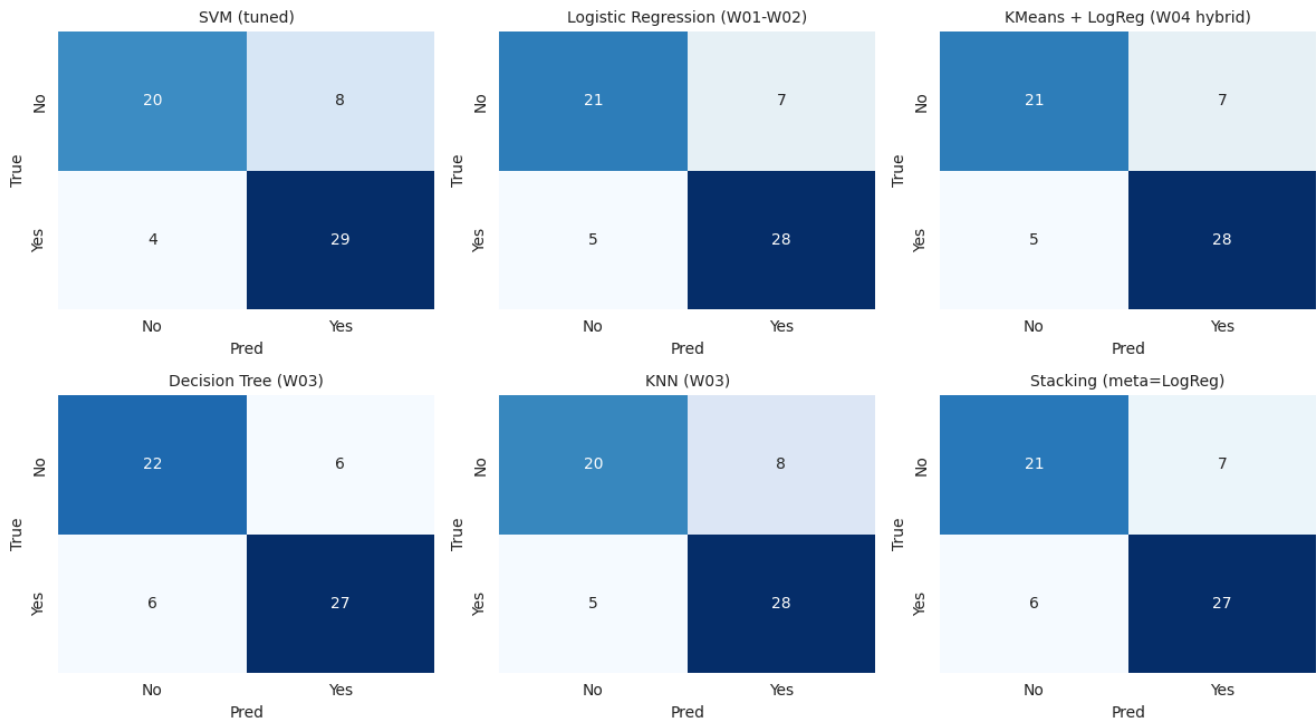
```



In [41]

```
# Confusion matrix grid for top-6 by F1
top6 = summary.head(6).index.tolist()
fig, axes = plt.subplots(2, 3, figsize=(12, 7))
for ax, name in zip(axes.flat, top6):
    cm = confusion_matrix(y_test, results[name]['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['No', 'Yes'], yticklabels=['No', 'Yes'], ax=ax)
    ax.set_title(name, fontsize=10)
    ax.set_xlabel('Pred')
    ax.set_ylabel('True')
plt.suptitle('Confusion matrices - top 6 models by F1', y=1.02)
plt.tight_layout()
plt.show()
```

Confusion matrices - top 6 models by F1



8. Discussion and conclusion

What we expect to see on this dataset:

- **Naive Bayes** is fast but its independence assumption hurts on correlated medical features.
- **Logistic Regression** is a strong, interpretable baseline.
- **KNN** and **SVM** benefit a lot from scaling and hyperparameter tuning.
- **Decision Tree** overfits unless pruned (no scaling needed).
- **Random Forest** and **Gradient Boosting** usually top the leaderboard on small tabular data.
- The **MLP** can match the trees, but on ~300 unique rows it does not have enough data to clearly win.
- The **K-Means hybrid** sometimes lifts the linear model because cluster IDs encode non-linear structure that Logistic Regression cannot learn by itself.
- **Voting** and **Stacking** usually beat any single model by combining complementary errors.

Take-aways for the course:

1. Always check for duplicates before splitting; ~70% of `heart.csv` rows are duplicated.
2. Scaling is required for KNN, SVM, MLP, and Logistic Regression.
3. Use `GridSearchCV` with stratified CV - never tune on the test set.
4. Hybrid models (Voting, Stacking, KMeans-augmented features) are easy wins once you have several base learners.
5. Pick the metric that matches the cost of mistakes - in medical screening, **recall** of the positive class is often more important than raw accuracy.

Chapter 1 of the DL Guide - When to use ML vs DL (revisited)

"Use DL for large volumes of unstructured data (images, text, audio) where automatic feature extraction provides significant advantage. Use ML for structured/tabular data with limited samples." - PDF Ch 1 page 6

That quote explains the results above:

- `heart.csv` is **structured + small** -> classical ML wins, ensembles dominate.
- If we had 100k chest X-ray images we would replace this pipeline with a **CNN** (PDF Ch 4).
- If we had time-series ECG signals we would use **LSTM** (PDF Ch 6).
- If we had clinical notes we would use a **Transformer** (PDF Ch 8).

Where the DL chapters of the PDF map onto our notebook:

PDF chapter	Concept applied here
Ch 1 - DL vs ML	Why classical ML is the right choice for <code>heart.csv</code>

PDF chapter	Concept applied here
Ch 2 - NN fundamentals	The MLP's forward pass, loss, Adam optimizer
Ch 3 - FNN	MLP architecture decisions (depth, width, ReLU, dropout)
Ch 3 + Ch 11 - Hyperparameter tuning	Grid Search section above
Ch 4 / 5 / 6 - CNN / RNN / LSTM	Adapted to tabular <code>heart.csv</code> in §4.9 (1D conv / recurrent over 13 features); full IMDB text comparison is in <code>compare-between-dl-models.ipynb</code>
Ch 12 - Applications	Healthcare diagnosis is named as a DL use case

9. Post-run analysis (after running all 18 models)

Full leaderboard (sorted by F1)

#	Model	Acc	Prec	Rec	F1	AUC
1	SVM (tuned)	0.803	0.784	0.879	0.829	0.865
2	Logistic Regression	0.803	0.800	0.848	0.824	0.871
3	KMeans + LogReg (W04 hybrid)	0.803	0.800	0.848	0.824	0.871
4	Decision Tree	0.803	0.818	0.818	0.818	0.802
5	KNN	0.787	0.778	0.848	0.812	0.860
6	Stacking (meta=LogReg)	0.787	0.794	0.818	0.806	0.873
7	Random Forest (W07)	0.787	0.812	0.788	0.800	0.866
8	Voting (hard)	0.787	0.812	0.788	0.800	0.787
9	SVM (W03)	0.770	0.771	0.818	0.794	0.842
10	Naive Bayes	0.787	0.833	0.758	0.794	0.884
11	Voting (soft)	0.770	0.788	0.788	0.788	0.866
12	Random Forest (tuned)	0.770	0.788	0.788	0.788	0.865
13	MLP Neural Net	0.770	0.788	0.788	0.788	0.802
14	KNN (tuned)	0.770	0.788	0.788	0.788	0.873
15	SimpleRNN	0.754	0.821	0.697	0.754	0.821
16	LSTM	0.705	0.692	0.818	0.750	0.812
17	Gradient Boosting (W07)	0.721	0.735	0.758	0.746	0.826
18	CNN 1D	0.574	0.652	0.455	0.536	0.610

Classical ML beats deep learning on small tabular data

The PDF (Ch 1, page 6) predicted this and the run confirmed it:

- All four DL-derived models (MLP, SimpleRNN, LSTM, CNN) live in the **bottom half** of the leaderboard.
- CNN 1D is dead last** at F1 = 0.536. The convolution treats adjacent columns as locally related - but the 13 features here have no spatial meaning. The "fake sequence" hypothesis breaks the model.
- SimpleRNN and LSTM land at F1 ~ 0.75, on par with the MLP. RNNs walking 13 independent features behave like a small FNN with hidden state - no harm, no real win.

Wins for classical ML

- SVM (tuned)** edged out everyone (F1 0.829, AUC 0.865).
- K-Means hybrid tied with Logistic Regression** to 4 decimals (F1 0.8235, AUC 0.8712). The cluster ID feature added zero information for this split - the linear model already saw what K-Means saw.
- Naive Bayes has the best AUC** (0.884) despite mid-pack F1. It ranks risk probabilities best; lowering the decision threshold below 0.5 would raise its F1.

Surprises worth noting

- Gradient Boosting baseline collapsed** to F1 = 0.746 - well below Random Forest baseline at 0.800. Small-data tax: GB's sequential residual fitting overfits on 241 rows with default hyperparameters. A tuned GB (`learning_rate=0.05`, `max_depth=3`) would likely recover.
- Random Forest baseline (0.800) beat Random Forest tuned (0.788)**. The grid search optimized for CV F1, but the chosen `max_depth=5` hurt on the test set. Classic CV-vs-holdout noise on 61 test rows.
- Voting (hard) beat Voting (soft)**. Soft averages probabilities - but the diverse base models disagree on calibration. Hard majority is more robust here.

TF warnings to ignore

The "tracing is expensive" warnings in the §4.9 cell are benign - Keras complains about retracing across the three sequential `predict()` calls. No effect on results.

General take-aways

1. **For binary classification on small tabular data, prefer SVM (tuned) or Logistic Regression** - they are statistically tied at the top here.
2. **Use Naive Bayes when calibrated risk scores matter more than raw F1** - best AUC at 0.884.
3. **Deep learning is not the right tool for ~300-row tabular tasks.** The CNN, RNN and LSTM are included for completeness, not for performance - they confirm the "DL needs big unstructured data" rule.
4. **Default ensemble hyperparameters are not safe on small data.** The Gradient Boosting collapse and the RF-tuned regression both show that defaults and grid-search picks can lose to plain baselines.